

31/8/26 DAY 8:

Smart Pointers

- Pointer that manages the lifetime of a dynamically allocated object.
- These are template classes that provide automatic memory management and help prevent memory leaks and dangling pointers.
- In order to use smart pointers, you need to include the `<memory>` header.

unique_ptr

- A smart pointer that owns and manages a dynamically allocated object.
- It ensures that there is only one unique_ptr pointing to the object at any given time.

Syntax:

```
unique_ptr<Type> ptr = std::make_unique<Type>(constructor_args); // Preferred way to create a unique_ptr
```

```
unique_ptr<Type> ptr(new Type(constructor_args)); // Not recommended, as it can lead to memory leaks if an exception is thrown before the unique_ptr takes ownership.
```

Example:

```
#include <iostream>
#include <memory>

int main() {
    unique_ptr<int> ptr = std::make_unique<int>(42);
    // auto ptr = std::make_unique<int>(42);
    std::cout << *ptr << std::endl;
    return 0;
}
```

Single Ownership

- A unique_ptr can only have one owner at a time.
- When the unique_ptr goes out of scope, it automatically deletes the managed object.
- The copy constructor and copy assignment operator are deleted for unique_ptr, preventing copying.
- As the name suggests, unique_ptr is unique and cannot be shared.

make_unique()

- A helper function that creates a `unique_ptr` and initializes it with the given arguments.
- It is the preferred way to create `unique_ptr` instances, as it provides exception safety and avoids potential memory leaks.
- It Returns a `unique_ptr` that owns the newly created object.

Automatic Resource Management

- `unique_ptr` automatically manages the lifetime of the object it owns. When the `unique_ptr` goes out of scope, it automatically deletes the managed object, preventing memory leaks.

Moving Ownership

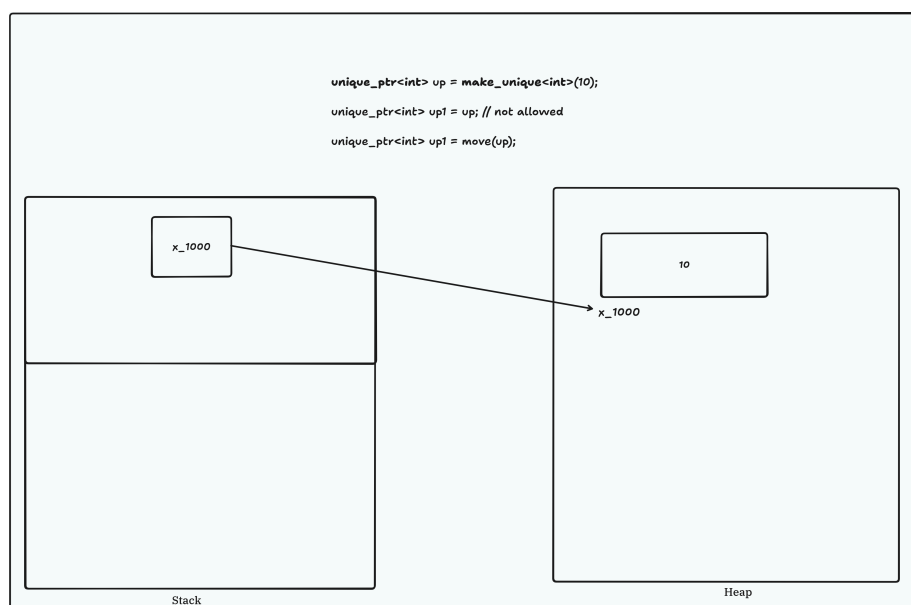
- `unique_ptr` supports move semantics, allowing ownership of the managed object to be transferred from one `unique_ptr` to another using `std::move()`. Example:

```
#include <iostream>
#include <memory>

int main() {
    unique_ptr<int> ptr1 = std::make_unique<int>(42);
    unique_ptr<int> ptr2 = std::move(ptr1); // Transfer ownership from ptr1 to ptr2

    if (!ptr1) {
        std::cout << "ptr1 is now null." << std::endl;
    }

    std::cout << *ptr2 << std::endl; // Output: 42
    return 0;
}
```



shared_ptr

- A smart pointer that allows multiple `shared_ptr` instances to share ownership of the same dynamically allocated object.
- It keeps track of the number of `shared_ptr` instances pointing to the same object using reference counting.
- Unlike `unique_ptr`, `shared_ptr` allows multiple owners for the same object.

Syntax:

```
shared_ptr<Type> ptr = std::make_shared<Type>(constructor_args); // Preferred way
to create a shared_ptr
```

```
shared_ptr<Type> ptr(new Type(constructor_args)); // Not recommended, as it can
lead to memory leaks if an exception is thrown before the shared_ptr takes
ownership.
```

Example:

```
#include <iostream>
#include <memory>

int main() {
    shared_ptr<int> ptr1 = std::make_shared<int>(42);

    std::cout << "Reference count: " << ptr1.use_count() << std::endl; // Output:
1
    shared_ptr<int> ptr2 = ptr1; // Both ptr1 and ptr2 share ownership of the same
object

    std::cout << *ptr1 << std::endl; // Output: 42
    std::cout << *ptr2 << std::endl; // Output: 42

    std::cout << "Reference count: " << ptr1.use_count() << std::endl; // Output:
2

    return 0;
}
```

Shared Ownership

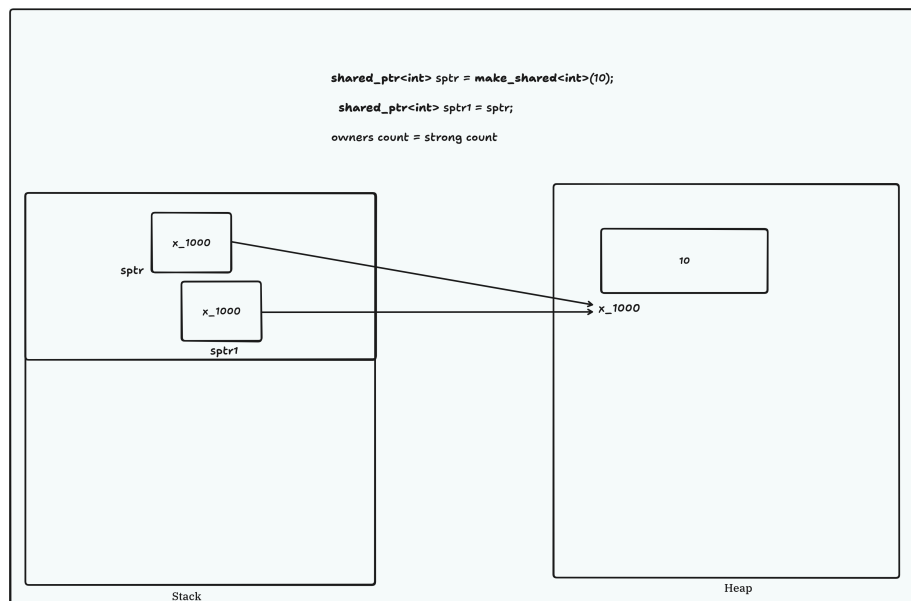
- `shared_ptr` allows multiple owners for the same object. When the last `shared_ptr` pointing to the object is destroyed or reset, the managed object is automatically deleted.
- The reference count is incremented when a new `shared_ptr` is created that points to the same object, and decremented when a `shared_ptr` is destroyed or reset.

Reference Counting

- `shared_ptr` uses reference counting to keep track of how many `shared_ptr` instances are pointing to the same object.
- When the reference count reaches zero, the managed object is deleted.
- Reference counting is a technique used to manage the lifetime of dynamically allocated objects.

make_shared()

- A helper function that creates a `shared_ptr` and initializes it with the given arguments.
- It is the preferred way to create `shared_ptr` instances, as it provides exception safety and avoids potential memory leaks.
- It Returns a `shared_ptr` that owns the newly created object.



weak_ptr

- A smart pointer that holds a non-owning reference to an object managed by `shared_ptr`.
- It is used to break ownership cycles and prevent memory leaks in situations where `shared_ptr` instances reference each other.

Syntax:

```
weak_ptr<Type> ptr = shared_ptr_instance; // Create a weak_ptr from a shared_ptr
```

Example:

```
#include <iostream>
#include <memory>

int main() {
    std::shared_ptr<int> ptr1 = std::make_shared<int>(42);

    std::cout << "Reference count: " << ptr1.use_count() << std::endl; // Output:
1
```

```

std::weak_ptr<int> ptr2 = ptr1; // Create a weak_ptr from a shared_ptr
std::cout << "Reference count: " << ptr1.use_count() << std::endl; // Output:
1

return 0;
}

```

Non-owning Reference

- A weak_ptr does not increase the reference count of the object it points to.
- It is used to observe an object without affecting its lifetime.

expired()

- The expired() method checks if the object that the weak_ptr points to is still alive.
- It returns true if the object is no longer available; otherwise, it returns false.

```

std::weak_ptr<int> weakPtr = sharedPtr;
if (weakPtr.expired()) {
    std::cout << "The object is no longer available." << std::endl;
} else {
    std::cout << "The object is still alive." << std::endl;
}

```

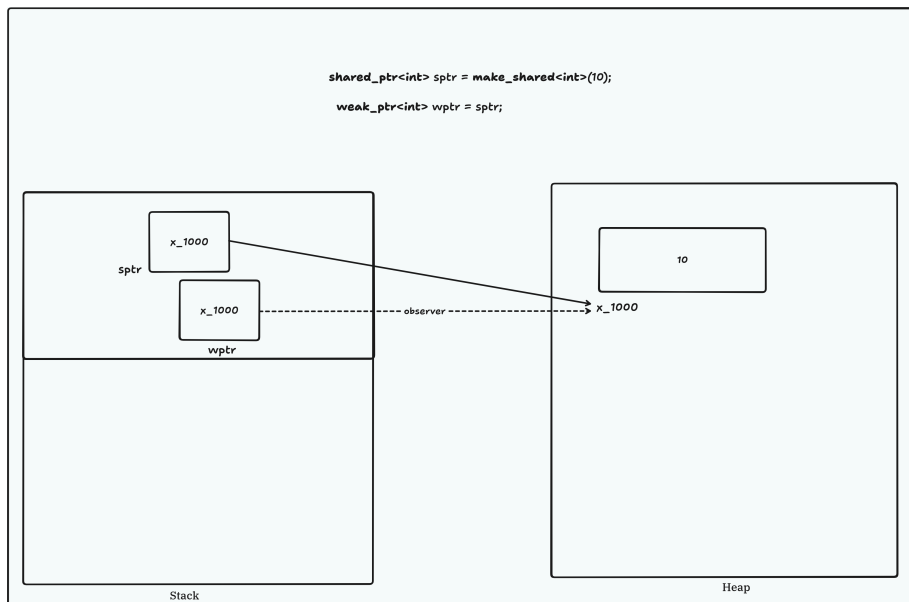
lock()

- The lock() method attempts to obtain a shared_ptr from a weak_ptr.
- If the object is still alive, it returns a valid shared_ptr; otherwise, it returns an empty shared_ptr.

```

if (auto sharedPtr = weakPtr.lock()) {
    // The object is still alive, and sharedPtr can be used to access it.
} else {
    // The object is no longer available.
}

```



Breaking Ownership Cycles

- Ownership cycles can occur when two or more `shared_ptr` instances reference each other, preventing the objects from being deleted.
- `weak_ptr` can be used to break ownership cycles by providing a non-owning reference to one of the objects in the cycle, allowing the other objects to be deleted when they are no longer needed.

```
#include <iostream>
#include <memory>
using namespace std;

class Child;

class Parent {
public:
    shared_ptr<Child> p;

    ~Parent() {
        cout << "Parent Destructor..." << endl;
    }
};

class Child {
public:
    shared_ptr<Parent> c; // Changed from weak_ptr to shared_ptr

    ~Child() {
        cout << "Child Destructor..." << endl;
    }
};

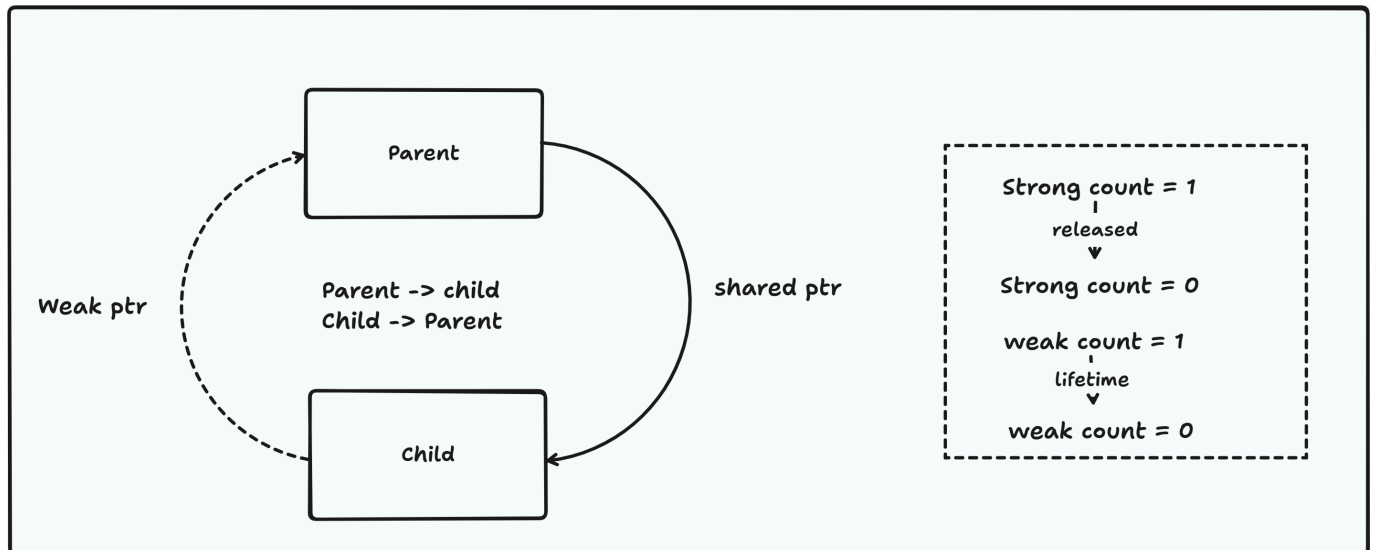
int main() {
    auto parent = make_shared<Parent>();
    auto child = make_shared<Child>();
```

```

// Create the circular dependency cycle
parent->p = child; // Parent points to Child (Child ref count = 2)
child->c = parent; // Child points to Parent (Parent ref count = 2)

cout << "Exiting main..." << endl;
return 0; // Local smart pointers leave scope here
}

```



Smart Pointer Comparison

Feature	unique_ptr	shared_ptr	weak_ptr
Ownership	Exclusive	Shared	Non-owning
Copy	Not allowed	Allowed	Not allowed
Move	Allowed	Allowed	Not allowed
Reference Count	None	Yes	Yes

Raw Pointer vs Smart Pointer

Raw Pointer	Smart Pointer
Manual memory management	Automatic memory management
No automatic cleanup	Automatic cleanup
Potential for memory leaks	No memory leaks
No ownership semantics	Clear ownership semantics
No reference counting	Reference counting (shared_ptr)
No safety features	Safety features (e.g., bounds checking)

Smart Pointers with Objects

- Smart pointers can be used to manage the lifetime of objects, ensuring proper cleanup and preventing memory leaks.
- When using smart pointers with objects, it is important to consider the ownership semantics and choose the appropriate smart pointer type (`unique_ptr`, `shared_ptr`, or `weak_ptr`) based on the desired ownership model.

Syntax:

```
unique_ptr<MyClass> ptr = std::make_unique<MyClass>(constructor_args);
```

```
auto ptr = make_unique<int>(42);
```

Example:

```
#include <iostream>
#include <memory>

class Student {
    int rollno;
public:
    Student(int id) : rollno(id) {
        std::cout << "Student Constructor..." << std::endl;
    }
    ~Student() {
        std::cout << "Student Destructor..." << std::endl;
    }
};

int main() {
    unique_ptr<Student> studentPtr = std::make_unique<Student>(12345);
    return 0;
}
```

Smart Pointers and Function Parameters

- Smart pointers can be passed as function parameters to transfer ownership or share ownership of dynamically allocated objects.
- When passing a `unique_ptr` to a function, it is typically passed by value or by reference to transfer ownership.
- When passing a `shared_ptr`, it can be passed by value to share ownership or by reference to avoid unnecessary reference count increments.

Different types of Function Parameters with Smart Pointers

1. **Passing by value:** The function takes ownership of the smart pointer, and the caller loses ownership.

```
void processStudent(unique_ptr<Student> studentPtr) {}  
void processStudent(shared_ptr<Student> studentPtr) {}
```

2. **Passing by reference:** The function can access the smart pointer without taking ownership, allowing the caller to retain ownership.

```
void processStudent(unique_ptr<Student>& studentPtr) {}  
void processStudent(const shared_ptr<Student>& studentPtr) {}  
void processStudent(const weak_ptr<Student>& studentPtr) {}
```

3. **Passing by pointer:** The function can access the smart pointer without taking ownership, allowing the caller to retain ownership.

```
void processStudent(Student* studentPtr) {}  
void processStudent(const Student* studentPtr) {}
```

4. **Passing by reference to the object:** The function can access the object without taking ownership of the smart pointer, allowing the caller to retain ownership.

```
void processStudent(Student& studentRef) {}  
void processStudent(const Student& studentRef) {}
```

Copy Constructor

- A special constructor that initializes a new object as a copy of an existing object.
- It is called when an object is passed by value, returned by value, or explicitly copied.

Syntax:

```
ClassName(const ClassName& other); // copy constructor syntax
```

Example:

```
#include <iostream>  
  
class Student {  
    int rollno;  
public:
```

```
Student(int id) : rollno(id) {
    std::cout << "Student Constructor..." << std::endl;
}
Student(const Student& other) : rollno(other.rollno) {
    std::cout << "Student Copy Constructor..." << std::endl;
}
~Student() {
    std::cout << "Student Destructor..." << std::endl;
}
};

int main() {
    Student student1(12345);
    Student student2 = student1; // copy constructor is called
    return 0;
}
```

Why Objects are Copied

- Objects are copied in C++ for several reasons, including:
 1. **Passing by Value:** When an object is passed to a function by value, a copy of the object is created to be used within the function.
 2. **Returning by Value:** When a function returns an object by value, a copy of the object is created to be returned to the caller.
 3. **Explicit Copying:** Objects can be explicitly copied using the copy constructor or copy assignment operator.
 4. **Container Classes:** When objects are stored in container classes (e.g., `std::vector`, `std::list`), they may be copied when the container resizes or when elements are inserted or removed.

Copy Constructor Invocation

The copy constructor is invoked in the same scenarios where objects are copied, such as:

1. **Passing by Value**
2. **Returning by Value**
3. **Explicit Copying**
4. **Container Classes**

Objects as Function Arguments

- When objects are passed as function arguments, they can be passed by value, by reference, or by pointer.
- The choice of passing method affects whether the copy constructor is invoked.

Syntax:

```
void functionName(ClassName obj); // Pass by value (copy constructor invoked)
void functionName(ClassName& obj); // Pass by reference (copy constructor not
```

```
invoked)
```

Returning Objects

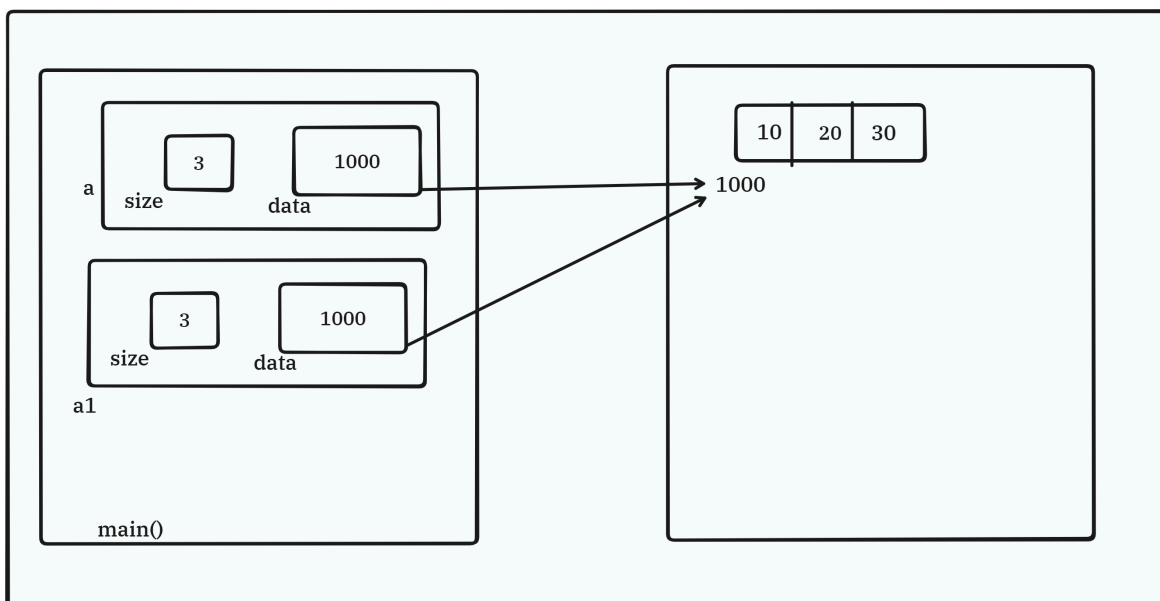
- When functions return objects by value, the copy constructor is invoked to create a copy of the object being returned.
- When functions return objects by reference or pointer, the copy constructor is not invoked.

Syntax:

```
ClassName functionName() {  
    ClassName obj;  
    return obj; // Copy constructor invoked  
}
```

Shallow Copy

- A shallow copy creates a new object that shares the same memory as the original object.
- It copies the values of the member variables, but if the member variables are pointers, both objects will point to the same memory location.
- This can lead to issues such as double deletion and dangling pointers.



Deep Copy

- A deep copy creates a new object with its own copy of the data, including dynamically allocated memory.
- It ensures that the new object has its own separate copy of the data, preventing issues related to shared memory and ownership.

Example:

```
class Array{
public:
    int* data;
    int size;

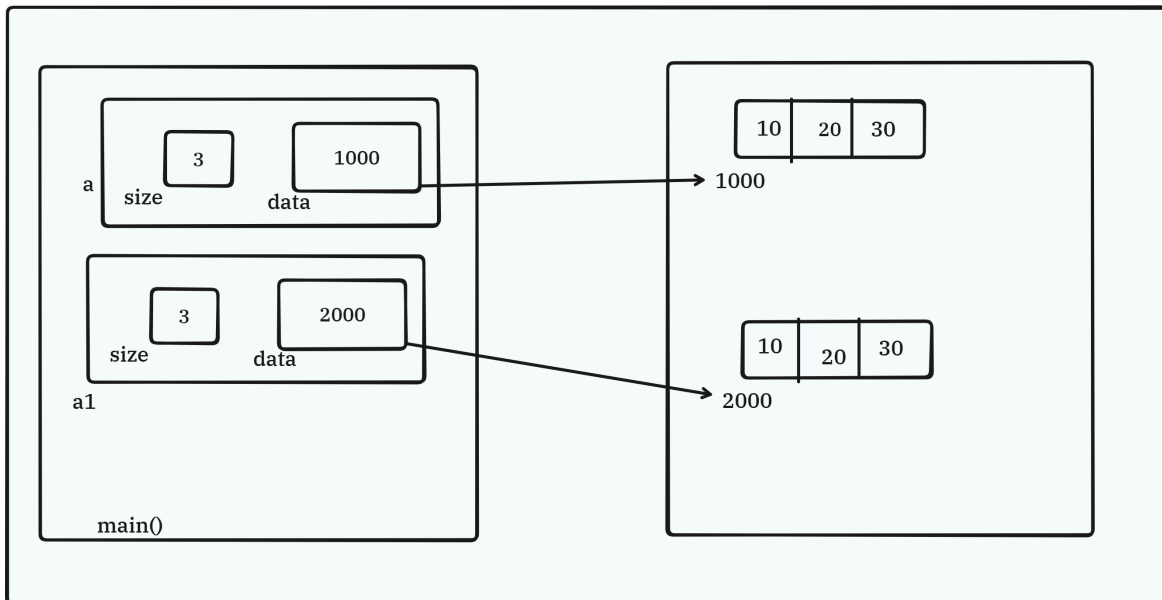
    Array() : data(nullptr), size(0){
        cout << "Default Constructor..." << endl;
    }
    // shallow copy: default nature for compiler
    // Array(const Array& a){
    //     cout << "Copy Constructor..." << endl;
    //     data = a.data; // data = value
    //     size = a.size;
    // }

    // Deep copy
    Array(const Array& a){
        cout << "Copy Constructor..." << endl;
        // data = a.data; // data = value
        size = a.size;
        data = new int[size];

        for(int i=0;i<size;i++){
            data[i] = a.data[i];
        }
    }

    Array(int size) : size(size){
        data = new int[size];
        cout << "Parameter Constructor..." << endl;
    }

    ~Array(){
        cout << "Destructor..." << endl;
        delete[] data;
    }
};
```



Tomorrow's Topic:

- Copy Assignment Operator
- Assignment Operator =
- Resource Ownership and Copying
- Rule of Three — Introduction
- move semantics